

SIMATS ENGINEERING

CO5-ASSESSMENT TOOL3-Reverse Engineering

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192511053
Name	B Santhosh Kumar

COURSE OUTCOME COVERED

CO5: Design intelligent agents and real-time AI-based systems (chatbots, expert systems, emotion detection, edge AI, autonomous drones, and related applications) by selecting appropriate agent architectures, knowledge representation, and reasoning mechanisms to solve real-world problems. (BL6)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Q1. The following is a conversation between a user and an AI-based flight-booking chatbot. Study the transcript given below and reverse-engineer the chatbot's design by: (a) writing its PEAS description, (b) classifying the type of agent it represents with justification, (c) identifying the likely component (NLU, Dialogue Manager, Knowledge Base, or NLG) responsible for each chatbot response, and (d) suggesting one enhancement using a learning-agent architecture.

Speaker	Message
User	I want to book a flight to Chennai next Friday.
Chatbot	Sure! What time would you like to depart, and do you have a preferred airline?
User	Morning flight, no preference.
Chatbot	I found 3 morning flights to Chennai on Friday. Shall I show you the cheapest option first?

Q2. The block diagram below shows the processing pipeline of a Real-Time Emotion Detection System. Reverse-engineer the system by: (a) explaining the function performed at each stage of the pipeline, (b) identifying the likely AI/ML technique used at each stage, (c) writing the PEAS description of the system, and (d) classifying the

Real-Time Emotion Detection System — processing pipeline



agent type (e.g., simple reflex, model-based) with justification.

Q3. An Expert System Shell for medical diagnosis contains the rule base, input facts, and reasoning trace given below. Study it and reverse-engineer the shell by: (a) identifying the type of chaining (forward/backward) used to reach the diagnosis, with justification, (b) identifying the expert-system-shell components illustrated (knowledge base, inference engine, explanation facility), (c) explaining how the shell would perform knowledge acquisition to add a new rule for dengue, and (d) explaining, with reasoning, whether backward chaining could reach the same diagnosis.

Rule	IF (condition)	THEN (conclusion)
R1	fever = yes AND cough = yes	suspect = flu
R2	suspect = flu AND body_ache = yes	diagnosis = Influenza
R3	fever = yes AND rash = yes	suspect = measles

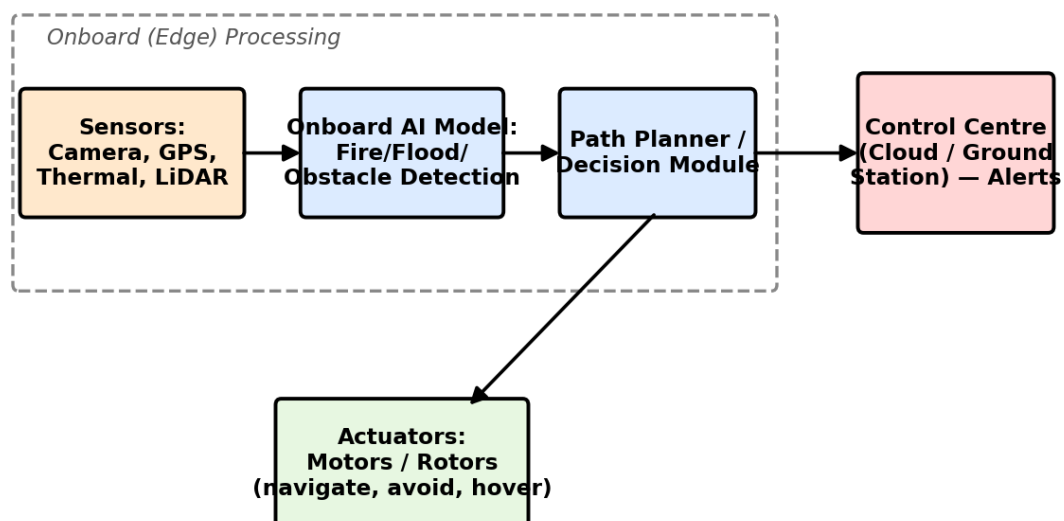
Given facts: fever = yes, cough = yes, body_ache = yes

Reasoning trace: R1 fires first (fever & cough matched) → suspect = flu. R2 then fires (suspect = flu & body_ache matched) → diagnosis = Influenza.

Explanation generated by the shell: “Diagnosis = Influenza because fever and cough indicated flu, and flu together with body ache confirmed Influenza.”

Q4. The diagram below shows the system architecture of an Autonomous Drone used for Disaster Detection with Edge AI deployment. Reverse-engineer the design by: (a) writing the PEAS description of the drone agent, (b) classifying the agent type (goal-based/utility-based) with justification, (c) identifying which components run on the edge versus the cloud and justifying why on-edge processing was chosen for detection and obstacle avoidance, and (d) identifying whether the agent's control architecture is reactive, deliberative, or hybrid, with justification.

Autonomous Drone — Disaster Detection (Edge AI) system architecture



Q5. The table below shows sample output logs from an AI-based Deepfake & Face Authenticity Verification system across five video frames. Study the output and reverse-engineer the system by: (a) identifying the likely input features used by the model (e.g., blink rate, texture/artifact score), (b) identifying the type of model/technique likely used to produce these outputs, (c) explaining how the final Real/Fake decision may have been derived from the frame-level confidence scores,

and (d) suggesting one way a swarm/ensemble-based approach could improve the system's reliability.

Frame No.	Blink Rate (per 10s)	Texture/Artifact Score	Fake Confidence (%)	Prediction
1	4	0.12	8	Real
2	1	0.61	74	Fake
3	0	0.83	91	Fake
4	5	0.09	5	Real
5	2	0.55	68	Fake

Overall system verdict: Fake (3 of 5 frames flagged — majority vote)

Code of Conduct:

*I **B Santhosh Kumar/192511053** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

B Santhosh Kumar

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning	Max Marks
Identification of Agent Type & PEAS (correctly inferred from the given artifact)	Correctly identifies the agent type and gives a complete, accurate PEAS description, fully consistent with the given artifact.	Identifies the agent type and PEAS description correctly with only minor omissions.	Partially correct identification of agent type/PEAS; some elements are missing or inconsistent with the artifact.	Agent type/PEAS identification is largely incorrect or not attempted.	10
Identification of Architecture & Components (mapping artifact evidence to system components)	Accurately identifies all relevant architectural components and correctly maps each to specific evidence in the artifact.	Identifies most components correctly with reasonable mapping to the artifact; minor gaps present.	Identifies some components; mapping to the artifact is weak or partially incorrect.	Little or no correct identification of architecture/components.	10
Identification of Underlying Technique/Algorithm/Reas	Correctly reverse-engineers the technique,	Correctly identifies the technique/	Identification is only partially correct or	Technique/mechanism is incorrectly	10

Criteria	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning	Max Marks
oning Mechanism	algorithm, or reasoning mechanism (e.g., chaining type, ML technique) with clear supporting evidence.	mechanism with adequate, mostly accurate justification.	lacks sufficient supporting evidence.	identified or not addressed.	
Quality of Justification & Use of Evidence (linking conclusions back to the given artifact)	Every conclusion is clearly justified with specific, accurate references to the given artifact.	Most conclusions are justified with reasonable references to the artifact.	Justification is vague or only loosely connected to the artifact.	Conclusions are stated with little or no justification or evidence.	10
Clarity & Completeness of Presentation (addresses all sub-parts of the question in an organised manner)	All sub-parts are addressed completely, in a clear, well-organised, and logically structured manner.	Most sub-parts are addressed clearly with minor gaps in organisation or completeness.	Some sub-parts are missing or the presentation is unclear/poorly organised.	Response is incomplete, disorganised, or largely missing.	10
				Total	50

Q1. AI-Based Flight-Booking Chatbot

Aim

To reverse-engineer the given flight-booking chatbot by identifying its **PEAS description, agent type, dialogue components, and a possible learning-agent enhancement.**

(a) PEAS Description

PEAS	Description
Performance Measure	Correct understanding of user requirements, relevant flight search, accurate recommendation and successful booking
Environment	User, flight database/API, airline schedules, fares and booking system
Actuators	Asking questions, displaying flights, recommending flights and confirming booking
Sensors	User messages and flight availability information

The chatbot receives the user's requirements through text and uses the available flight information to provide a suitable option. The transcript specifically shows the user giving the destination, date and preferred time.

(b) Type of Agent

Goal-Based Agent

The chatbot is a **goal-based agent** because its main goal is to help the user complete a flight-booking task.

Evidence from conversation	Interpretation
"book a flight to Chennai next Friday"	Establishes the user's goal
Asks for departure time and airline	Identifies missing information
"Morning flight, no preference"	Updates the available requirements

Finds 3 morning flights	Uses the information to move towards the goal
-------------------------	---

The reasoning can be represented as:

User requirement → Collect missing information → Search flights →

Recommend option

Therefore, the chatbot is best classified as a **goal-based agent with dialogue-state tracking**.

(c) Components Responsible for Responses

Chatbot Response 1

"Sure! What time would you like to depart, and do you have a preferred airline?"

Component	Role
NLU	Extracts flight-booking intent, Chennai and next Friday
Dialogue Manager	Detects that departure time and airline are missing
NLG	Converts the selected response into natural language

Flow:

User message → NLU → Dialogue Manager → NLG → Response

Chatbot Response 2

"I found 3 morning flights to Chennai on Friday. Shall I show you the cheapest option first?"

Component	Role
NLU	Identifies morning preference and no airline preference
Knowledge Base / Flight API	Provides available morning flights
Dialogue Manager	Decides the next action/recommendation
NLG	Produces the final response

The phrase "**I found 3 morning flights**" indicates that the system has obtained information from a flight data source, while the cheapest-option suggestion indicates a dialogue decision.

(d) Learning-Agent Enhancement

A useful enhancement is to make the chatbot learn from the user's previous choices.

For example:

Previous behaviour	Learned preference
Frequently selects morning flights	Rank morning flights higher
Usually selects cheaper flights	Give low-cost options higher priority
Often chooses direct flights	Prefer direct flights

Learning flow

User choice → Recommendation → Accept/Reject → Feedback → Update preference → Better future recommendation

This would reduce repeated questions and make the recommendations more personalised.

Conclusion

The given chatbot is a **goal-based task-oriented agent**. Its architecture can be represented as:

NLU → Dialogue Manager → Flight Knowledge/API → NLG

A learning component can further improve the system by learning the user's flight preferences.

Q2. Real-Time Emotion Detection System

Aim

To reverse-engineer the given emotion-detection pipeline by explaining each processing stage, identifying the likely AI techniques, constructing its PEAS description and classifying the agent.

The given pipeline is:

Camera → Face Detection → Feature Extraction (CNN) → Emotion Classifier → Emotion Label + Confidence

(a) Function of Each Stage

Stage	Function
Camera	Captures live video frames
Face Detection	Locates the face in each frame
Feature Extraction	Extracts useful facial patterns; the diagram explicitly specifies CNN
Emotion Classifier	Classifies the detected facial expression
Output	Produces emotion label and confidence

For example:

Camera frame → Face region → CNN features → Emotion class → Confidence

Possible emotion outputs are **Happy, Sad, Angry, Fear, Surprise and Neutral**.

(b) Likely AI/ML Technique

Stage	Likely technique
Camera	Image/video acquisition
Face Detection	Face-detection algorithm such as Haar Cascade or CNN-based detector

Feature Extraction	CNN — explicitly given in the diagram
Emotion Classification	Supervised multiclass classification
Output	Predicted class with confidence score

The most certain identification is **CNN**, because it is directly written in the given artifact. The exact face-detection algorithm and classifier are not specified, so they should be treated as likely implementations rather than definite ones.

(c) PEAS Description

PEAS	Description
Performance	Accurate emotion detection, fast response and low false prediction
Environment	Person, lighting, background, camera position and surroundings
Actuators	Display emotion, confidence and application-specific alert
Sensors	Camera/video frames

(d) Agent Type

Simple Reflex Agent with a Learned Perception Model

The system responds mainly to the **current camera frame**.

Current frame → Face detection → Emotion classification → Output

There is no evidence in the given diagram of long-term memory, planning or future-state reasoning. Therefore, a simple reflex classification is more appropriate than a goal-based or deliberative agent.

The CNN is the **learned perception model**, but this alone does not make the complete system a model-based agent.

Conclusion

The system is a **real-time perception-based emotion detector**, using a CNN for feature extraction and a classifier for emotion recognition.

Q3. Expert System Shell for Medical Diagnosis

Aim

To reverse-engineer the medical expert-system shell by identifying its **chaining method, shell components, knowledge-acquisition process and possibility of backward reasoning**.

The given facts are:

fever = yes, cough = yes, body_ache = yes

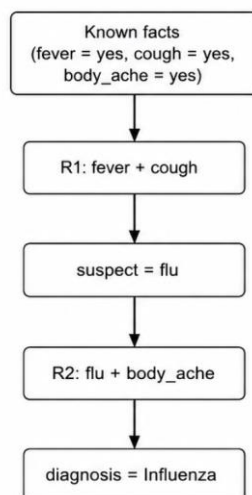
The trace shows **R1 → R2 → Influenza**.

(a) Type of Chaining

Forward Chaining

The system starts with known facts and moves towards the diagnosis.

Reasoning



R1 produces the new fact **suspect = flu**. This newly derived fact is then used by R2. Therefore, the reasoning is clearly **forward chaining**.

(b) Expert-System Shell Components

Evidence in artifact	Component	Function
R1, R2, R3 rule table	Knowledge Base	Stores diagnostic rules

R1 and R2 firing sequence	Inference Engine	Matches facts with rules and derives conclusions
Generated explanation	Explanation Facility	Explains why Influenza was concluded

Knowledge Base

The rules represent stored diagnostic knowledge:

- **R1:** fever + cough → flu
- **R2:** flu + body ache → Influenza
- **R3:** fever + rash → measles

Inference Engine

The engine performs:

R1 fires → suspect = flu

then:

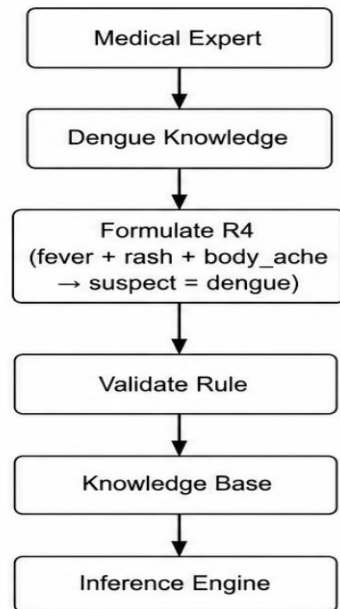
R2 fires → diagnosis = Influenza

(c) Knowledge Acquisition – Adding Dengue Rule

Proposed Rule

Rule	IF condition	THEN conclusion
R4	fever = yes AND rash = yes AND body_ache = yes	suspect = dengue

Knowledge Acquisition Process

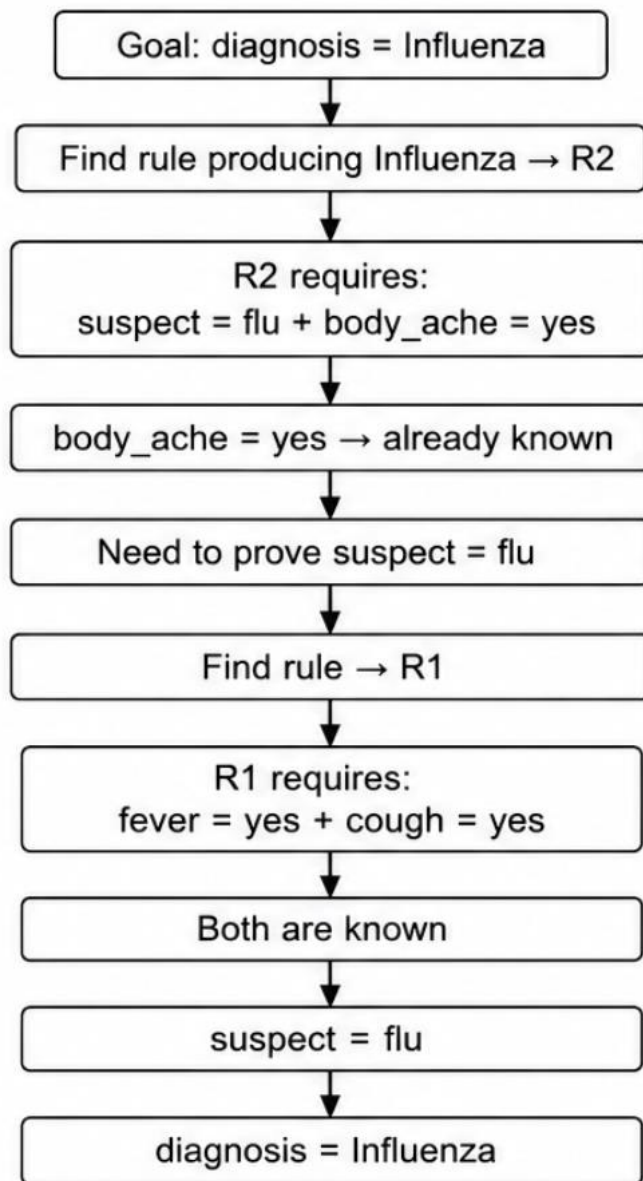


Step	Activity
1. Collect knowledge	Obtain dengue-related knowledge from a domain expert
2. Formulate rule	Convert it into the existing IF–THEN format
3. Validate	Check the rule with a qualified expert
4. Update KB	Add validated R4 to the knowledge base
5. Test	Give suitable facts and check whether R4 fires

(d) Can Backward Chaining Reach the Same Diagnosis?

Yes.

Backward chaining starts with the goal:



Forward Chaining	Backward Chaining
Starts with facts	Starts with goal
R1 fires first	R2 is considered first
Derives suspect = flu	Tries to prove suspect = flu
Reaches Influenza	Reaches Influenza

Conclusion

Both methods can reach **diagnosis = Influenza** in this example because the required facts are available. The difference is only the **direction of reasoning**.

Q4. Autonomous Drone for Disaster Detection

Aim

To reverse-engineer the autonomous disaster-detection drone by identifying its PEAS, agent type, edge/cloud components and control architecture.

The given architecture contains **sensors, onboard AI, path planner, control centre and actuators**.

(a) PEAS Description

PEAS	Drone System
Performance	Accurate disaster detection, safe navigation, fast response and efficient battery use
Environment	Disaster areas, buildings, roads, weather, people and obstacles
Actuators	Motors/rotors for navigation, hovering and obstacle avoidance
Sensors	Camera, GPS, thermal sensor and LiDAR

(b) Agent Type

Utility-Based Agent

The drone has to balance several factors instead of simply reaching a destination.

For example:

Factor	Requirement
Safety	Avoid dangerous areas
Distance	Prefer efficient routes
Battery	Reduce unnecessary energy consumption
Response time	Reach affected area quickly
Detection	Cover important disaster locations

Suppose two routes are available:

Route A: shorter but dangerous

Route B: slightly longer but safer

A utility-based agent can choose **Route B** if its overall utility is higher.

Therefore, the drone is best classified as a **utility-based agent**.

(c) Edge vs Cloud

Edge Processing

The diagram places the following on the drone:

- Camera
- GPS
- Thermal sensor
- LiDAR
- Onboard AI model
- Fire/flood/obstacle detection
- Path planner/decision module

Cloud / Ground

The diagram shows:

Control Centre (Cloud / Ground Station) → Alerts

Why use Edge AI?

The main reason is **low latency**.

For example:

LiDAR detects obstacle → Edge AI → Decision → Motor

This can happen locally without waiting for cloud processing.

Edge processing	Advantage
Obstacle detection	Immediate response
Fire/flood detection	Faster local decision
Path planning	Less network dependency
Local processing	Lower communication delay

Therefore, critical detection and obstacle avoidance are placed on the edge.

(d) Control Architecture

Hybrid Architecture

The system contains both reactive and deliberative behaviour.

Reactive part

Obstacle detected → Immediately change movement

This is suitable for fast obstacle avoidance.

Deliberative part

The **Path Planner / Decision Module** considers the destination, hazards and route before deciding the movement.

Therefore:

Reactive obstacle avoidance + Deliberative path planning = Hybrid architecture

Conclusion

The drone is a **utility-based agent with a hybrid control architecture**, where time-critical AI processing is performed on the edge and alerts are communicated to the cloud/ground control centre.

Q5. Deepfake & Face Authenticity Verification

Aim

To reverse-engineer the deepfake detection system using the given five-frame output log and determine its features, likely model, final decision mechanism and an ensemble-based improvement.

The supplied artifact gives five frames and explicitly states the overall verdict as **Fake, with 3 of 5 frames flagged.**

(a) Likely Input Features

The table directly provides two features:

Feature	Purpose
Blink Rate	Measures blinking behaviour
Texture/Artifact Score	Represents visual irregularities associated with manipulation

Other useful features could include:

- facial landmarks,
- eye movement,
- mouth movement,
- face-boundary artifacts,
- lighting consistency,
- temporal consistency between frames.

The last group is a possible extension because it is **not explicitly given in the table.**

(b) Likely Model / Technique

The output contains two classes:

Real / Fake

and a **Fake Confidence (%)**.

Therefore, the most reasonable interpretation is:

Supervised Binary Classification

A possible architecture is:

**Face/Video Features → Feature Extraction → Binary Classifier →
Real/Fake + Confidence**

A CNN could be used for visual feature extraction, but the exact model cannot be confirmed from the given table.

Therefore, the safest conclusion is:

The system likely uses a supervised binary classification model, possibly supported by CNN-based visual feature extraction.

(c) Final Real/Fake Decision

Frame-Level Analysis

Frame	Fake Confidence	Prediction	Evidence
1	8%	Real	Very low fake evidence
2	74%	Fake	Strong fake evidence
3	91%	Fake	Strongest fake evidence
4	5%	Real	Very low fake evidence
5	68%	Fake	Strong fake evidence

Prediction Count

Prediction	Frames	Percentage
Fake	3	60%
Real	2	40%

Therefore:

3 Fake > 2 Real

Final Verdict

Fake

This exactly agrees with the overall verdict provided in the artifact: **3 of 5 frames flagged — majority vote.**

Confidence Evidence

Fake frame	Fake confidence
Frame 2	74%
Frame 3	91%
Frame 5	68%
Average	77.67%

The Real-predicted frames have only **8% and 5%** fake confidence.

Hence, the frame-level evidence strongly supports the Fake classification.

Important observation

If all five confidence values are simply averaged:

[

$$\frac{8+74+91+5+68}{5}=49.2\%$$

]

So a simple 50% average threshold would not produce Fake.

Method	Result
Majority vote	Fake
Simple average = 49.2%	Real if threshold = 50%
Weighted aggregation	Could produce Fake

Therefore, the most defensible explanation is that the final verdict was obtained by **majority voting**, as explicitly stated in the artifact, rather than by simply averaging all confidence values.

(d) Ensemble / Swarm-Based Improvement

Instead of relying on one detector, multiple detectors can examine different aspects of the video.

Detector	Checks
Detector 1	Facial texture
Detector 2	Blink/eye behaviour
Detector 3	Temporal consistency
Detector 4	Face-boundary artifacts

Their outputs can then be combined:

Detector 1 + Detector 2 + Detector 3 + Detector 4

↓

Ensemble Decision

↓

Final Real/Fake Result